

Równoważne programy - omówienie

XXIII OI, etap III
Dzień próbny

Kod zadania: row
Limit pamięci: 128 MiB



Treść

Mamy dane dwa ciągi liczb całkowitych z przedziału $[1, k]$, oba o długości n ($1 \leq n \leq 10^5, 1 \leq k \leq 1000$). Naszym celem jest stwierdzenie, czy przekształcenie pierwszego ciągu w drugi jest możliwe. Możemy zamieniać miejscami dwa sąsiednie elementy pierwszego ciągu, ale tylko dla niektórych par wartości.

Optymalna strategia

Będziemy "budować" kolejne prefiksy drugiego ciągu. Zaczniemy od wyszukania jakiegoś i spełniającego warunek $a_i = b_1$, po czym przesuniemy go na początek, zostawiając resztę nietkniętą. Zapomnimy teraz o istnieniu a_i oraz b_1 . Powtórzymy ten krok dla kolejnych b_i póki będzie to możliwe (musimy przerwać jeśli nie ma już szukanej wartości lub nie da się przenieść elementu na początek).

Jak w optymalny sposób wybierać które a_i przesuwamy w danym kroku? Odpowiedź jest bardzo prosta: wystarczy brać pierwszy od lewej, gdyż wzięcie jakiegoś dalszego wystąpienia danej wartości nie pomaga nam w żaden sposób.

Teraz pozostaje nam problem efektywnej implementacji naszego pomysłu. Chcielibyśmy umieć wykonywać 3 operacje:

1. znajdź pierwsze wystąpienie danej wartości
2. podaj zbiór wartości występujący na danym prefiksie
3. usuń dany indeks

Wzorcówka

Nie będziemy konkatelować indeksów (zezwolimy na "dziury" w naszym ciągu a). Pierwsza operacja jest bardzo łatwa w obsłudze, wystarczy dla każdej wartości trzymać stos jej indeksów w kolejności malejącej i usuwać z wierzchu. Zauważmy, że k jest stosunkowo małe. Spróbujmy trzymać ograniczenia nieprzemienności i kompozycje prefiksów jako maski bitowe (bitsety). Te pierwsze będziemy mogli wyznaczyć już na wejściu. Te drugie będą podlegały zmianom, więc będziemy je trzymać jako drzewo przedziałowe.

Dokładniej: jeżeli jakiś element już usunęliśmy, to jego liść w drzewie przedziałowym ustawiony jest na pusty bitset. W przeciwnym przypadku na bitset z jedynym zapalonym bitem w a_i . W węzłach będziemy utrzymywać bitset będący OR-em jego synów.

Jak teraz uporać się z operacjami 2 i 3? W operacji 2 robimy zapytanie o OR bitsetów na pewnym prefiksie, po czym patrzymy czy bitset możliwych zamian jest jego podmaską. W operacji 3 wystarczy wyzerować dany bitset (w liściu) i zaktualizować drzewo.

Nasz algorytm jest efektywny, bo operacje na bitsetach o długości k są tu bardzo wydajne ($\frac{k}{64} < 16$). Ostateczna złożoność to $O\left(\frac{n \cdot k \cdot \log_2 n}{64}\right)$. Co ciekawe, jest to w teorii lepsze od $O(n \cdot k)$ które jest w oficjalnym rozwiązaniu.

